

```
sent_creation_tx = bdb.transactions.send_commit(fulfilled_
creation_tx)
```

Step 7: Now we will prepare the transfer of the asset ownership from Mike to Sarah.

```
txid = fulfilled_creation_tx['id']
asset_id = txid
transfer_asset = {
    'id': asset_id
}
output_index = 0
output = fulfilled_creation_tx['outputs'][output_index]
transfer_input = {
    'fulfillment': output['condition']['details'],
    'fulfills': {
        'output_index': output_index,
        'transaction_id': fulfilled_creation_tx['id']
    },
    'owners_before': output['public_keys']
}
prepared_transfer_tx = bdb.transactions.prepare(
    operation='TRANSFER',
    asset=transfer_asset,
    inputs=transfer_input,
    recipients=sarah.public_key,
)
fulfilled_transfer_tx = bdb.transactions.fulfill(
    prepared_transfer_tx,
    private_keys=mike.private_key,
)
```

Step 8: Finally, we send the commit to the database to complete the TRANSFER transaction.

```
sent_transfer_tx = bdb.transactions.send_commit(fulfilled_
transfer_tx)
```

Step 9: Let us confirm that the transactions went through as planned and the ownership has been assigned correctly from Mike to Sarah.

```
print("Is Sarah the owner of the painting?",
    sent_transfer_tx['outputs'][0]['public_keys'][0] ==
sarah.public_key)
print("Was Mike the previous owner of the painting?",
    fulfilled_transfer_tx['inputs'][0]['owners_before'][0] ==
mike.public_key)
```

Use cases for BigchainDB

As discussed, BigchainDB treats every node in its database as an independent asset. This opens up room for a wide

array of applications for it. Some of those use cases have been listed below.

1. BigchainDB can be used for registering claims for assets such as vehicles and real-estate properties.
2. It can be used for modelling and designing scalable NFT platforms.
3. BigchainDB is great for token creation and distribution due to its support for divisible assets. This means that multiple assets can point to a single asset in BigchainDB, making it ideal for token launches.
4. BigchainDB can be used as a version control system for volatile and sensitive assets such as documents.
5. It can be used for storage of time series data, although there are better alternatives available for that purpose.
6. BigchainDB is excellent for creating state machines, as it is ideal for marking state changes using the accessible metadata feature.

Alternatives to using BigchainDB

As far as emulating the functionality of blockchain technology is concerned, BigchainDB stands one of one for now. Currently, there are no direct alternatives to using BigchainDB in the sense that there are no other databases that provide all the features exactly as this database has to offer. However, there are a few indirect alternatives worth mentioning.

- First up is Cassandra. While Cassandra is a NoSQL database, it has a few blockchain emulating features that could potentially make it an ideal alternative. It uses distributed computing techniques which makes your data highly secure. There are no single points of failure. Cassandra is also fault-tolerant due to its replicating feature across multiple data centres and has extremely low latency. What's more, it's an open source Apache project.
- ChainifyDB is a proprietary alternative to BigchainDB in that it offers a cloud based blockchain service using which you can secure your local database.
- Postchain can also be a good alternative if the solution you are looking for is focused more on creating a blockchain rather than a blockchain-like database. Postchain is a framework for creating blockchains that provides deeper integration with SQL databases. 

 By: Mir H.S. Quadri

The author is a research analyst with a specialisation in artificial intelligence and machine learning. He is a senior content developer at Arkinotel. He shares a deep love for analysis of technological trends and understanding its implications. Being a FOSS enthusiast, he has contributed to several open source projects.